



Escuela Politécnica Nacional
Ingeniería en Software
Aplicaciones Móviles



Integrantes:

- Daniel Arias.
- Erick Caicedo.
- Belén Cholango.

Taller grupal

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo Nativo Puro (iOS/Android)

Es el estándar de oro en rendimiento y acceso a hardware. Se basa en hablar el "idioma materno" del sistema operativo.

- **Tecnologías:** Swift/Objective-C (iOS) y Kotlin/Java (Android).
- **Funcionamiento:** El código se compila directamente a binarios que el procesador entiende. Usa los componentes de UI que el propio SO provee.
- **Relevancia: Máxima.** Sigue siendo la opción obligatoria para apps que requieren máximo rendimiento (juegos pesados, edición de video) o funciones de sistema muy profundas.
- **Detalle clave:** La llegada de frameworks declarativos como **SwiftUI** y **Jetpack Compose** ha modernizado el nativo, permitiendo escribir interfaces de forma mucho más rápida y similar a la web.

2. La Era del WebView: Apps dentro de Navegadores

Apareció como la solución "barata" y rápida: meter una página web dentro de una app.

- **Tecnologías:** HTML, CSS, JavaScript. Frameworks: **PhoneGap (Adobe/Cordova)** e **Ionic** (en sus inicios).
- **Funcionamiento:** La app es esencialmente un navegador minimalista (WebView) que renderiza la web. Un "puente" permitía acceder a funciones limitadas como la cámara.
- **Relevancia: Baja/Nicho.** PhoneGap fue descontinuado. Ionic evolucionó (Capacitor) para ser mucho más eficiente, pero el concepto de "WebView puro" ha sido superado por su falta de fluidez.
- **Detalle clave:** Su gran pecado era la "latencia de entrada" y que el scroll nunca se sentía realmente natural.

3. El Puente (Bridge): El Renacimiento de JavaScript

Aquí es donde React Native cambió el juego. No es una web, es código JavaScript que "comanda" componentes nativos.

- **Tecnologías:** JavaScript / TypeScript.

- **Funcionamiento:** Existe un "Puente" (Bridge) que comunica el hilo de JavaScript con el hilo nativo. Si mueves un botón en JS, el puente le dice al sistema: "Oye, dibuja este botón nativo allá".
- **Relevancia: Muy Alta.** Es el estándar en la industria para apps corporativas y redes sociales (Instagram, Airbnb).
- **Detalle clave:** Su limitación es el cuello de botella del puente. Si hay demasiada información pasando por él (como animaciones complejas), la app se ralentiza. Por eso, React Native está migrando a **JSI (JavaScript Interface)** para eliminar ese puente.

4. Renderizado Propio: El Modelo Skia

Flutter rompió el paradigma del puente. En lugar de pedirle al SO que dibuje un botón, Flutter trae su propio "estuche de pinturas".

- **Tecnologías:** Dart (Lenguaje) y Skia/Impeller (Motores gráficos).
- **Funcionamiento:** Flutter funciona como un motor de videojuegos. Dibuja cada píxel en la pantalla de forma independiente a los componentes del SO. Esto garantiza que la app se vea idéntica en Android e iOS.
- **Relevancia: Muy Alta.** Es la tecnología de mayor crecimiento por su velocidad de desarrollo y belleza visual.
- **Detalle clave:** Al no usar componentes nativos, a veces la integración con funciones muy específicas del sistema (como un teclado muy personalizado) puede ser más trabajosa.

5. El Futuro: Nuevas Dimensiones y Desafíos

Foldables (Plegables)

Aquí la arquitectura debe ser agnóstica al tamaño.

- **Desafío:** El "Continuity Mode". El software debe manejar el cambio de $\$DPI\$$ y resolución en milisegundos.
- **Estado del arte:** Jetpack Compose y SwiftUI ya manejan "Adaptive Layouts", pero el reto es el **Estado de la Aplicación**. Se requiere que la lógica de negocio (ViewModels) sea totalmente independiente de la UI para que el "re-render" no borre lo que el usuario estaba escribiendo.

AR/XR (Realidad Extendida)

Se pasó de coordenadas $\$(x, y)\$$ a $\$(x, y, z)\$$.

- **Limitación de Cross-Platform:** React Native y Flutter sufren aquí porque el renderizado 3D requiere una comunicación directa con la GPU para evitar el mareo del usuario (latencia).
- **La solución actual:** Se usan motores como **Unity** o **Unreal Engine** con integraciones nativas (C++), o herramientas propietarias como **RealityKit** (Apple) para asegurar que el objeto virtual se mantenga anclado a la realidad sin retraso.

Wearables

- **La clave:** Eficiencia energética extrema. Aquí el desarrollo tiende a volver al **Nativo Puro** porque cada ciclo de CPU ahorrado se traduce en minutos de vida de batería para el reloj.

Resumen de Relevancia (2026):

Tecnología	Caso de Uso Ideal	Relevancia Actual
Nativo (Kotlin/Swift)	Apps de alto rendimiento, AR, Wearables.	Crítica
Flutter	Desarrollo rápido, UI personalizada, Multiplataforma.	Muy Alta
React Native	Apps de contenido, equipos con base web (JS).	Muy Alta
Capacitor / Ionic	MVPs rápidos, apps internas empresariales simples.	Media

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

Caso de estudio: Uber y Telegram.

1. Ingeniería Inversa de la Decisión

- **Uber y el Framework RIBs:** Uber no solo desarrolla en nativo (Swift/Kotlin), sino que creó su propia arquitectura llamada RIBs.

¿Por qué una empresa de transporte invertiría millones de dólares en crear una arquitectura propia en lugar de usar las estándar (como MVVM o VIPER)? ¿Qué relación tiene esto con el hecho de tener cientos de ingenieros tocando el mismo código simultáneamente?

Uber no usa las arquitecturas estándar (como MVVM) porque a su escala, esas arquitecturas fallan en el aislamiento de fallos. Uber tiene cientos de ingenieros trabajando en la misma app. Si todos modifican el mismo "ViewController", la app se vuelve inestable.

RIBs (Router, Interactor, Builder): Es una arquitectura orientada a estados y jerarquía de lógica, no a pantallas. Permite que el equipo que trabaja en "Uber Eats" y el que trabaja en "Uber Rides" operen en la misma app sin que el código de uno rompa el del otro. Es una decisión de escalabilidad de equipo.

- **Telegram y el Enfoque "Desde Cero":** Telegram es famoso por su extrema velocidad y ligereza. A diferencia de otras apps que usan librerías estándar, Telegram desarrolla casi todo in-house (incluso su propio protocolo de comunicación, MTProto).

¿Por qué para una app de mensajería la eficiencia del binario (el peso de la app) y la gestión de memoria manual son más críticas que la velocidad de desarrollo?

Telegram prioriza la eficiencia de recursos sobre la velocidad de desarrollo. Al escribir su propio protocolo (MTProto) y usar C++ para casi todo el procesamiento de datos, logran que la app sea increíblemente rápida incluso en teléfonos de hace 10 años con poca RAM.

Mientras WhatsApp o Messenger dependen de librerías del sistema que pueden ser pesadas, Telegram optimiza el binario para que la transferencia de datos sea mínima, lo que es vital en países con conexiones a internet inestables.

2. Análisis de Rendimiento y Arquitectura

- **El Desafío de la Geolocalización en Uber:** Uber necesita mantener una conexión por socket constante, calcular rutas en tiempo real y actualizar la posición de múltiples vehículos en un mapa.

Desde una perspectiva de gestión de hilos (threading) y consumo de batería, ¿por qué el desarrollo nativo es la única opción viable para evitar que el sistema operativo "mate" la aplicación mientras el usuario espera su viaje?

El problema de Uber no es solo mostrar un mapa, es el ciclo de vida del proceso.

Gestión de hilos: Uber debe procesar coordenadas de GPS, señales de red y cálculos de ruta en hilos de fondo (Background Threads). Los frameworks multiplataforma a veces sufren latencia al despertar estos hilos.

Batería: El acceso nativo a las APIs de ubicación (CoreLocation en iOS / Google Play Services en Android) permite usar el hardware de forma "fina", pidiendo actualizaciones de ubicación solo cuando el acelerómetro detecta movimiento, algo que es mucho más complejo y menos eficiente de tunear a través de un puente (Bridge).

- **La Gestión de Memoria en Telegram:** Telegram maneja cantidades masivas de caché (fotos, videos, stickers).

¿Cómo beneficia el uso de lenguajes como C++ (a través de JNI en Android) para la manipulación de archivos y cifrado, en lugar de depender enteramente de la máquina virtual de Java/Kotlin?

Telegram es básicamente un gestor de archivos masivo.

Uso de C++ (NDK/C++ Interop): Al usar C++, Telegram gestiona la memoria de forma manual. Esto evita el "Garbage Collector" (recolector de basura) de Java/Kotlin, que a veces causa pequeños tirones (lags) en la interfaz cuando se liberan muchos datos a la vez. Por eso se puede hacer scroll en Telegram a través de gigabytes de fotos y stickers sin que la app se detenga ni un milisegundo.

3. El Desafío de las Nuevas Pantallas

Tanto Uber como Telegram dependen de la confianza del sistema operativo. Si pasamos a dispositivos plegables donde la app debe dividirse en dos (multitarea activa):

¿Cómo maneja la arquitectura nativa el preservado de estado (que no se reinicie el mapa de Uber al doblar el teléfono) comparado con un framework que corre sobre una capa de abstracción?

¿Qué ventaja tiene Telegram al usar componentes nativos de UI para adaptarse instantáneamente a nuevos factores de forma (aspect ratio) sin deformar la interfaz?

En el caso de dispositivos plegables, el Desarrollo Nativo tiene una ventaja absoluta por la Gestión de Configuración.

Preservado de Estado: Cuando doblas un teléfono, el sistema operativo Android técnicamente "destruye" y "recrea" la actividad. Uber, al ser nativo, puede usar onSaveInstanceState de forma profunda para que el mapa no se reinicie y el usuario no pierda de vista dónde viene su auto.

Adaptación de UI: Telegram utiliza componentes nativos que responden a las "Window Insets" (las medidas de la ventana). Al no tener un motor de renderizado propio (como Flutter), el sistema operativo le dicta a la app exactamente cómo estirarse, y Telegram solo tiene que ajustar sus márgenes.

4. Veredicto Crítico: Uber vs. Telegram

Si tuvieras que construir una aplicación de Streaming de Video en vivo con baja latencia (estilo Twitch) que debe funcionar perfectamente en el Apple Watch y en autos inteligentes (Android Auto/CarPlay):

¿Elegirías la arquitectura altamente modular y desacoplada de Uber (RIBs) o el enfoque de máximo rendimiento y optimización de recursos de Telegram? Fundamenta basándote en la escalabilidad del equipo vs. la escalabilidad del rendimiento.

El enfoque de Telegram es la mejor opción por el factor “máximo rendimiento”.

Fundamentación Arquitectónica:

1. **Baja Latencia:** El streaming de video requiere decodificación de hardware en tiempo real. La arquitectura de Telegram, que permite insertar módulos de C++, es superior para manejar códecs de video (\$H.264\$ / \$H.265\$) con mínima latencia.
2. **Multidispositivo (Watch/Car):** Un reloj inteligente tiene memoria y batería extremadamente limitadas. La arquitectura pesada de Uber (RIBs) añadiría demasiada sobrecarga. El enfoque de Telegram de "hacerlo ligero y nativo" permite que la app corra fluida incluso en procesadores muy pequeños como los de un Apple Watch.

Pregunta final:

¿Qué es una aplicación nativa, hay varios, como se clasifican?

Un aplicativo nativo es aquel que se construye utilizando el Software Development Kit (SDK) oficial y los lenguajes designados por el creador del sistema operativo. Esto permite que la app hable directamente con el núcleo (kernel) del teléfono sin intermediarios.

Se clasifican principalmente en tres categorías:

A. Según el Paradigma de Programación

- **Imperativo (Tradicional):** El programador describe el camino para llegar a un estado (ej. "Crea un botón, ponlo en el centro, cámbiale el color a azul").
Ejemplos: Android con Java/Kotlin (XML) o iOS con UIKit.
- **Declarativo (Moderno):** El programador describe cómo debe verse la interfaz según el estado (ej. "Si el usuario está logueado, muestra este botón azul"). El sistema se encarga del resto.
Ejemplos: Jetpack Compose (Android) y SwiftUI (iOS).

B. Según el Nivel de Acceso

- **Nativo de Alto Nivel:** Utiliza las librerías estándar de la interfaz de usuario del sistema (botones, tablas, listas oficiales). Es lo más común.
- **Nativo de Bajo Nivel (C++ / NDK):** Se usa para tareas que requieren potencia bruta, como el motor de cifrado de Telegram o el procesamiento de mapas en 3D de Uber. Se programa directamente sobre los drivers de gráficos o procesamiento.

C. Según su Propósito de Ejecución

- **Foreground Apps:** Apps con las que interactúas directamente (la mayoría).
- **Background Services / Headless:** Apps nativas que no tienen interfaz pero controlan procesos críticos (como el servicio de GPS que sigue rastreando tu ubicación en Uber aunque bloques el teléfono).

